

```

Checking the commit readiness of the chaincode definition successful on peer0.org2 on channel 'mychannel'
Using organization 2
+ peer lifecycle chaincode approveformyorg -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /home/berlinflx/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem --channelID mychannel --name basic --version 1.0 --package-id basic_1.0:1d4d445573112813889f87c307bc2b5f6e664c87b24c035bee15bcc7f59b629 --sequence 1
+ res=0
2026-04-08 13:43:06.689 IST 0001 INFO [chaincodeCmd] ClientWait -> txid [9173d0f9e6f76a9893f79e914d52bc76bc1eebeaf485079545670e144dbcc591] committed with status (VALID) at localhost:9051
Chaincode definition approved on peer0.org2 on channel 'mychannel'
Using organization 1
Checking the commit readiness of the chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to check the commit readiness of the chaincode definition on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode checkcommitreadiness --channelID mychannel --name basic --version 1.0 --sequence 1 --output json
+ res=0
{
  "approvals": {
    "Org1MSP": true,
    "Org2MSP": true
  }
}
Checking the commit readiness of the chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Checking the commit readiness of the chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to check the commit readiness of the chaincode definition on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode checkcommitreadiness --channelID mychannel --name basic --version 1.0 --sequence 1 --output json
+ res=0
{
  "approvals": {
    "Org1MSP": true,
    "Org2MSP": true
  }
}
Checking the commit readiness of the chaincode definition successful on peer0.org2 on channel 'mychannel'
Using organization 1
Using organization 2
+ peer lifecycle chaincode commit -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /home/berlinflx/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem --channelID mychannel --name basic --peerAddresses localhost:7051 --tlsRootCertFiles /home/berlinflx/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem --peerAddresses localhost:9051 --tlsRootCertFiles /home/berlinflx/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem --version 1.0 --sequence 1
+ res=0
2026-04-08 13:43:14.998 IST 0001 INFO [chaincodeCmd] ClientWait -> txid [55ecbbd1f511fe6e3e4dbc1129565e3f1d6959f671a3a57cfff6dfccb67ea25d7] committed with status (VALID) at localhost:9051
2026-04-08 13:43:15.008 IST 0002 INFO [chaincodeCmd] ClientWait -> txid [55ecbbd1f511fe6e3e4dbc1129565e3f1d6959f671a3a57cfff6dfccb67ea25d7] committed with status (VALID) at localhost:7051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...

```

Figure 2: Chaincode approved by both organisations and committed to the channel

Accessing the environment

Before using the peer CLI directly, export the variables that tell it which node to talk to and which identity to use. Run these inside the WSL terminal from the test-network directory:

```

export PATH="$HOME/bin:$HOME/fabric-samples/bin:$PATH"
export FABRIC_CFG_PATH="$HOME/fabric-samples/config/"
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_ADDRESS=localhost:7051

export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp

ORDERER_CA=$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlsacerts/tlsca.example.com-cert.pem

```

```

ORG1_CA=$HOME/fabric-samples/test-network/organizations/peerOrganizations/
org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
ORG2_CA=$HOME/fabric-samples/test-network/organizations/peerOrganizations/
org2.example.com/peers/peer0.org2.example.com/tls/ca.crt

```

 **Note:** these variables configure the CLI as the Org1 Admin identity on peer0.org1 (port 7051). Every invoke passes both `--peerAddresses` flags so that Org1 and Org2 both endorse the transaction; this is required by the default endorsement policy.

Running the experiments

Experiment 1 — Create an asset and verify it: Initialise the ledger with six sample assets, and then create a custom one assigned to Suyash:

```

peer chaincode invoke -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com --tls
  --cafile $ORDERER_CA \
  -C mychannel -n basic \
  --peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_CA \

```